

JAVA PROGRAMMING ASSIGNMENT 13

Problem 1: Car Attributes Management

- **Description:** Develop a `Car` class with attributes like `make`, `model`, `year`, and `price`. Include methods to set and get these attributes. The program will create multiple `Car` objects, modify their attributes, and then display their details.
- **Solution Approach:** Implement the `Car` class with required methods. Instantiate multiple cars, modify their attributes using setters, and display each car's details using getters or a custom method designed for display.
- **Test Cases:**
 - **Test Case 1:** Modify the `year` and `price` of a car and verify the update.
 - **Test Case 2:** Attempt to set a negative value for `price` and handle it gracefully.
 - **Test Case 3:** Check the output when no modifications are made to a new car's attributes.

Problem 2: Library Book Management

- **Description:** Create a `Book` class with attributes `title`, `author`, `isbn`, and a boolean `availabilityStatus`. The `Library` class will manage these books, including functionalities to add, check out, and return books, as well as display their status.
- **Solution Approach:** Define the `Book` class with methods to manage status. The `Library` class should have a collection (like `ArrayList`) of books. Implement methods in the `Library` class to manipulate the books collection.
- **Test Cases:**
 - **Test Case 1:** Add a book and then check it out. Verify that the `availabilityStatus` changes correctly.
 - **Test Case 2:** Return a checked-out book and verify its status.
 - **Test Case 3:** Attempt to check out a book that's already checked out and handle the error.

Problem 3: Employee Information System

- **Description:** Design an `Employee` class with attributes for `name`, `employeeId`, `department`, and `salary`. This program will handle multiple employee objects, allowing updates to their salaries and displaying their information.
- **Solution Approach:** Create the `Employee` class, instantiate multiple employees, provide a method to update salary, and another to display employee details. Ensure all attributes can be modified appropriately.
- **Test Cases:**
 - **Test Case 1:** Update an employee's salary and verify the change.
 - **Test Case 2:** Update to an invalid salary (e.g., negative numbers) and check error handling.
 - **Test Case 3:** Display details of employees to ensure all attributes are shown correctly.

Problem 4: Student Gradebook

- **Description:** Construct a `Student` class with `name`, `studentId`, and an array of `grades`. The `Gradebook` class will manage a list of these students, allowing the addition of students, updating grades, and computing average grades.
- **Solution Approach:** Develop the `Student` class with methods to manage grades. The `Gradebook` class should handle a collection of students, with methods to add and update students and their grades.
- **Test Cases:**
 - **Test Case 1:** Add grades to a student and calculate the average.
 - **Test Case 2:** Handle edge cases like no grades or all identical grades.
 - **Test Case 3:** Verify the accuracy of grade updates and average calculations.

Problem 5: Product Inventory

- **Description:** Implement a `Product` class with `productId`, `productName`, `quantityInStock`, and `pricePerUnit`. An `Inventory` class will manage these products, including functionalities to add products, update stock quantities, and calculate the total value of the inventory.
- **Solution Approach:** Build the `Product` class with necessary methods. The `Inventory` class should handle a collection of products, with methods to update stock and calculate inventory values.
- **Test Cases:**
 - **Test Case 1:** Add products and update the quantity. Verify that the total stock and value are updated correctly.
 - **Test Case 2:** Reduce the stock quantity and ensure the inventory decreases properly.
 - **Test Case 3:** Calculate the total value of the inventory and verify against manual calculations.